LSTM Weekly Rain Model - Kaggle Training Notebook

Notebook nay ghi toan bo source code vao `/kaggle/working/lstm_rain_weekly/` roi chay `main.py` de huan luyen mo hinh LSTM **Direct Multi-step** du bao luong mua (`total_precipitation`) theo gio cho ca **168 gio (1 tuan)** tiep theo tai cac diem luoi nam trong lanh tho dat lien Viet Nam, bao gom diem sat bien gioi, sat bo bien va cac diem noi dia.

Yeu cau: them dataset Parquet tai `/kaggle/input/datasets/nguyentrangggggg/vietnam-meteorological-weather-data-parquet/weather.parquet` , them GeoPackage GADM level-0 tai `/kaggle/input/datasets/nglan271204/vnm-gpkg/gadm41_VNM.gpkg` va bat **GPU T4** trong Settings.

In [ ]:
```
!pip install -q seaborn geopandas shapely pyogrio
```

In [ ]:
```
import os

os.makedirs('/kaggle/working/data', exist_ok=True)
os.makedirs('/kaggle/working/lstm_rain_weekly', exist_ok=True)
os.makedirs('/kaggle/working/lstm_rain_weekly/data', exist_ok=True)
os.makedirs('/kaggle/working/lstm_rain_weekly/preprocessing', exist_ok=True)
os.makedirs('/kaggle/working/lstm_rain_weekly/dataset', exist_ok=True)
os.makedirs('/kaggle/working/lstm_rain_weekly/model', exist_ok=True)
os.makedirs('/kaggle/working/lstm_rain_weekly/training', exist_ok=True)
os.makedirs('/kaggle/working/lstm_rain_weekly/visualization', exist_ok=True)

print('All directories created.')
```

In [ ]:
```
%%writefile /kaggle/working/lstm_rain_weekly/data/__init__.py
# package marker
```

In [ ]:
```
%%writefile /kaggle/working/lstm_rain_weekly/data/load_data.py
"""
Nap du lieu thoi tiet cho cac diem luoi nam trong lanh tho Viet Nam.

Luon loc toa do truoc khi doc day du du lieu:
  1. Chi doc hai cot latitude/longitude de lay danh sach toa do duy nhat.
  2. Dung GeoPackage GADM level-0 de giu cac toa do nam trong polygon Viet Nam
     da buffer 0.05 do.
  3. Moi doc day du _COLS tu Parquet voi filters theo cac toa do hop le.
  4. Sap xep chuoi thoi gian va ep float64 -> float32.
"""
from pathlib import Path
from typing import Any, cast

import geopandas as gpd
```

```python
import pandas as pd
from shapely.geometry import Point

DATA_PATH = '/kaggle/input/datasets/nguyentrangggggg/vietnam-meteorological-weather-
GPKG_PATH = '/kaggle/input/datasets/nglan271204/vnm-gpkg/gadm41_VNM.gpkg'
LOCAL_GPKG_PATH = './data/gadm41_VNM.gpkg'
LAND_BUFFER_DEGREES = 0.05

target_cols = ['total_precipitation']

_COORD_COLS = ['latitude', 'longitude']

_COLS = [
    'latitude', 'longitude', 'valid_time',
    'temperature_celsius', 'apparent_temperature',
    'relative_humidity', 'wind_speed', 'wind_direction',
    'total_precipitation', 'total_cloud_cover',
    'mean_sea_level_pressure', 'surface_pressure',
    'sea_surface_temperature', 'air_density',
]


def _resolve_gpkg_path(gpkg_path: str) -> str:
    """Uu tien duong dan Kaggle, fallback ve ./data khi test local."""
    if Path(gpkg_path).exists():
        return gpkg_path
    if Path(LOCAL_GPKG_PATH).exists():
        return LOCAL_GPKG_PATH
    return gpkg_path


def _union_geometries(geometry):
    """Gop tat ca polygon Viet Nam thanh mot hinh hoc duy nhat."""
    return geometry.union_all()


def _load_vietnam_land_geometry(gpkg_path: str = GPKG_PATH):
    """Doc polygon quoc gia Viet Nam tu GeoPackage GADM level-0."""
    gpkg_path = _resolve_gpkg_path(gpkg_path)
    print(f"Dang doc ranh gioi Viet Nam tu GeoPackage: {gpkg_path}")
    vietnam_level0 = gpd.read_file(gpkg_path, layer=0)

    if vietnam_level0.empty:
        raise ValueError("GeoPackage khong co hinh hoc Viet Nam o layer=0.")

    # Du lieu thoi tiet la kinh/vi do, nen dua polygon ve EPSG:4326 neu can.
    if vietnam_level0.crs is not None and vietnam_level0.crs.to_epsg() != 4326:
        vietnam_level0 = vietnam_level0.to_crs(epsg=4326)

    vietnam_geom = _union_geometries(vietnam_level0.geometry)

    if vietnam_geom.is_empty:
        raise ValueError("Khong tao duoc polygon Viet Nam tu GeoPackage.")

    return vietnam_geom
```

```python
def _read_unique_coordinates(data_path: str) -> pd.DataFrame:
    """
    Chi doc hai cot toa do tu Parquet.

    Buoc nay khong doc cac cot khi tuong lon nhu nhiet do/mua/ap suat, nen RAM
    nhe hon rat nhieu so voi doc full _COLS truoc roi moi loc.
    """
    print("Dang doc nhe 2 cot latitude/longitude de lay toa do duy nhat ...")
    coords = pd.read_parquet(data_path, columns=_COORD_COLS, engine='auto')
    unique_coords = coords.drop_duplicates().reset_index(drop=True)
    print(f"So diem toa do duy nhat truoc khi loc: {len(unique_coords):,}")
    return unique_coords


def _find_vietnam_land_coordinates(
    unique_coords: pd.DataFrame,
    vietnam_geom,
    buffer_degrees: float = LAND_BUFFER_DEGREES,
) -> pd.DataFrame:
    """
    Loc toa do nam trong polygon Viet Nam da buffer.

    Chi tao Point va contains() cho danh sach toa do duy nhat, tuyet doi khong
    chay phep toan hinh hoc tren hang trieu dong du lieu thoi gian.
    """
    vietnam_area = vietnam_geom.buffer(buffer_degrees)

    # Shapely Point dung thu tu (x, y) = (longitude, latitude).
    points = [
        Point(lon, lat)
        for lat, lon in unique_coords[_COORD_COLS].itertuples(index=False, name=Non
    ]
    inside_mask = [vietnam_area.contains(point) for point in points]

    land_coords = unique_coords.loc[inside_mask, _COORD_COLS].copy()
    print(
        "So diem toa do nam trong lanh tho Viet Nam sau khi loc "
        f"(buffer={buffer_degrees} do): {len(land_coords):,}"
    )

    if land_coords.empty:
        raise ValueError(
            "Khong co diem luoi nao nam trong polygon Viet Nam. "
            "Hay kiem tra CRS/toa do hoac duong dan GeoPackage."
        )

    return land_coords


def _build_coordinate_filters(keep_coords: pd.DataFrame) -> list:
    """
    Tao Parquet filters theo cap toa do hop le.

    Dung OR theo tung latitude, trong moi latitude dung longitude in [...]
    de filter dung cap (latitude, longitude) nhung ngan gon hon hang nghin
```

```
    dieu kien OR rieng le.
    """
    coords = keep_coords[_COORD_COLS].astype('float64')
    lon_by_lat: dict[float, list[float]] = {}

    for lat_value, lon_value in coords.itertuples(index=False, name=None):
        lat = float(cast(Any, lat_value))
        lon = float(cast(Any, lon_value))
        lon_by_lat.setdefault(lat, []).append(lon)

    return [
        [
            ('latitude', '==', lat),
            ('longitude', 'in', lons),
        ]
        for lat, lons in lon_by_lat.items()
    ]


def _read_filtered_weather_data(data_path: str, keep_coords: pd.DataFrame) -> pd.Da
    """Doc day du _COLS chi cho cac toa do dat lien da duoc loc truoc."""
    filters = _build_coordinate_filters(keep_coords)
    print(f"Dang doc Parquet voi filters cho {len(keep_coords):,} diem toa do hop l
    return pd.read_parquet(
        data_path,
        columns=_COLS,
        engine='auto',
        filters=filters,
    )


def load_data(
    data_path: str = DATA_PATH,
    gpkg_path: str = GPKG_PATH,
    buffer_degrees: float = LAND_BUFFER_DEGREES,
) -> pd.DataFrame:
    vietnam_geom = _load_vietnam_land_geometry(gpkg_path)
    unique_coords = _read_unique_coordinates(data_path)
    land_coords = _find_vietnam_land_coordinates(unique_coords, vietnam_geom, buffe
    df = _read_filtered_weather_data(data_path, land_coords)

    # valid_time phai la datetime de tao feature thoi gian dung ve sau.
    df['valid_time'] = pd.to_datetime(df['valid_time'])

    # Sap xep de moi cap toa do co chuoi thoi gian lien tuc truoc khi tao sequence.
    df = df.sort_values(['latitude', 'longitude', 'valid_time']).reset_index(drop=T

    # float64 -> float32 giup giam RAM khi feature engineering va train LSTM.
    for col in df.select_dtypes('float64').columns:
        df[col] = df[col].astype('float32')

    n_locs = df.groupby(['latitude', 'longitude'], sort=False).ngroups
    print(
        "Nap du lieu dat lien Viet Nam thanh cong! "
        f"{len(df):,} dong | {n_locs:,} diem toa do | target_cols={target_cols}"
```

```
    )
    return df
```

In [ ]:
```
%%writefile /kaggle/working/lstm_rain_weekly/preprocessing/__init__.py
# package marker
```

In [ ]:
```
%%writefile /kaggle/working/lstm_rain_weekly/preprocessing/feature_engineering.py
"""
Task: Build the feature matrix for weekly rainfall forecasting.

Pipeline order (each step depends on the previous one):
  1. Interpolate raw meteorological NaN (per coordinate group).
  2. log1p transform of total_precipitation — rainfall is extremely right-skewed
     (lots of zeros, rare heavy bursts); log1p compresses the tail so the LSTM
     trains on a near-symmetric target.
  3. YEARLY feature   : 'year_normalized' = (year - 2020) / 6  → long-term climate
  4. MONTHLY feature  : 'monthly_mean_precipitation' = historical mean rainfall per
                        (lat, lon, month) → a climatology baseline telling the mod
                        which months are the rainy season of each region.
  5. WEEKLY context   : lag (1, 3, 6, 24, 168h) and rolling-mean (3, 6, 24, 168h)
                        features → the model "remembers" the past week.
  6. Cyclic time encodings (hour, day_of_week, month, day_of_year as sin/cos).
  7. Drop residual NaN rows created by the lag/rolling boundaries.

IMPORTANT: every rainfall-derived feature below is computed AFTER the log1p
transform, so the model and all baselines live in a single consistent log space.
"""
import numpy as np
import pandas as pd

TARGET_COL = 'total_precipitation'

_METEO_COLS = [
    'temperature_celsius', 'apparent_temperature',
    'relative_humidity', 'wind_speed', 'wind_direction',
    'total_precipitation', 'total_cloud_cover',
    'mean_sea_level_pressure', 'surface_pressure',
    'sea_surface_temperature', 'air_density',
]

_LAG_HOURS  = [1, 3, 6, 24, 168]
_ROLL_HOURS = [3, 6, 24, 168]


# ── 1. Interpolate raw NaN ──────────────────────────────────────────────

def _interpolate_raw(df: pd.DataFrame) -> pd.DataFrame:
    existing = [c for c in _METEO_COLS if c in df.columns]
    df[existing] = (
        df.groupby(['latitude', 'longitude'], sort=False)[existing]
        .transform(lambda x: x.interpolate(method='linear', limit_direction='both'))
    )
    return df
```

```python
# ── 2. log1p transform of rainfall ──────────────────────────────────────

def _log_transform_target(df: pd.DataFrame) -> pd.DataFrame:
    # Clip tiny negative noise to 0 before log1p (precip can never be negative).
    df[TARGET_COL] = np.log1p(df[TARGET_COL].clip(lower=0)).astype('float32')
    return df


# ── 3. Yearly climate-drift feature ─────────────────────────────────────

def _add_year_feature(df: pd.DataFrame) -> pd.DataFrame:
    df['year_normalized'] = ((df['valid_time'].dt.year - 2020) / 6).astype('float32
    return df


# ── 4. Monthly climatology baseline ─────────────────────────────────────

def _add_monthly_climatology(df: pd.DataFrame) -> pd.DataFrame:
    month = df['valid_time'].dt.month
    df['monthly_mean_precipitation'] = (
        df.groupby(['latitude', 'longitude', month], sort=False)[TARGET_COL]
        .transform('mean')
        .astype('float32')
    )
    return df


# ── 5. Weekly context: lags + rolling means ─────────────────────────────

def _add_lag_features(df: pd.DataFrame) -> pd.DataFrame:
    grp = df.groupby(['latitude', 'longitude'], sort=False)[TARGET_COL]
    for h in _LAG_HOURS:
        df[f'{TARGET_COL}_lag{h}'] = grp.shift(h).astype('float32')
    return df


def _add_rolling_features(df: pd.DataFrame) -> pd.DataFrame:
    grp = df.groupby(['latitude', 'longitude'], sort=False)[TARGET_COL]
    for h in _ROLL_HOURS:
        # shift(1) guarantees the current hour never leaks into its own feature.
        df[f'{TARGET_COL}_roll{h}'] = (
            grp.transform(lambda x: x.shift(1).rolling(h, min_periods=1).mean())
            .astype('float32')
        )
    return df


# ── 6. Cyclic time encodings ────────────────────────────────────────────

def _add_time_features(df: pd.DataFrame) -> pd.DataFrame:
    dt = df['valid_time']
    df['hour_sin']        = np.sin(2 * np.pi * dt.dt.hour      / 24 ).astype('float
    df['hour_cos']        = np.cos(2 * np.pi * dt.dt.hour      / 24 ).astype('float
    df['day_of_week_sin'] = np.sin(2 * np.pi * dt.dt.dayofweek / 7  ).astype('float
    df['day_of_week_cos'] = np.cos(2 * np.pi * dt.dt.dayofweek / 7  ).astype('float
    df['month_sin']       = np.sin(2 * np.pi * dt.dt.month     / 12 ).astype('float
```

```
    df['month_cos']       = np.cos(2 * np.pi * dt.dt.month     / 12 ).astype('float
    df['day_of_year_sin'] = np.sin(2 * np.pi * dt.dt.dayofyear / 365).astype('float
    df['day_of_year_cos'] = np.cos(2 * np.pi * dt.dt.dayofyear / 365).astype('float
    return df


# ── Orchestrator ──────────────────────────────────────────────────────────

def engineer_features(df: pd.DataFrame) -> pd.DataFrame:
    df = _interpolate_raw(df)
    df = _log_transform_target(df)
    df = _add_year_feature(df)
    df = _add_monthly_climatology(df)
    df = _add_lag_features(df)
    df = _add_rolling_features(df)
    df = _add_time_features(df)
    df = df.dropna().reset_index(drop=True)
    return df
```

In [ ]:

```
%%writefile /kaggle/working/lstm_rain_weekly/preprocessing/scaling.py
"""
Task: Fit a MinMaxScaler on all numeric feature columns, scale the DataFrame
      in-place to [0, 1], and persist both the scaler and the feature-column
      list to disk so inference can reproduce the exact transform.
"""
import pickle
import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

SCALER_PATH       = 'scaler_rain.pkl'
FEATURE_COLS_PATH = 'feature_cols_rain.pkl'

# Coordinates / time are identifiers, not model inputs → never scaled.
_EXCLUDE = {'latitude', 'longitude', 'valid_time'}


def fit_and_scale(df: pd.DataFrame):
    feature_cols = [c for c in df.columns if c not in _EXCLUDE]

    scaler = MinMaxScaler(feature_range=(0, 1))
    scaled = scaler.fit_transform(df[feature_cols].values.astype(np.float32))
    df[feature_cols] = scaled.astype(np.float32)

    with open(SCALER_PATH, 'wb') as f:
        pickle.dump(scaler, f)
    with open(FEATURE_COLS_PATH, 'wb') as f:
        pickle.dump(feature_cols, f)

    print(f"  Scaler          -> {SCALER_PATH}")
    print(f"  Feature columns -> {FEATURE_COLS_PATH}  ({len(feature_cols)} cols)")

    return df, scaler, feature_cols
```

In [ ]:
```
%%writefile /kaggle/working/lstm_rain_weekly/dataset/__init__.py
# package marker
```

In [ ]:
```
%%writefile /kaggle/working/lstm_rain_weekly/dataset/sequence_dataset.py
"""
Task: Turn the scaled DataFrame into per-coordinate numpy arrays and build
      lazy-loading sequence indices, split temporally per coordinate
      (80% train / 20% test) so there is zero geographic leakage.

Forecasting setup (Direct Multi-step):
  X : 168 past hours  × N features   → shape (168, N)
  y : 168 future hours of rainfall   → shape (168,)
      i.e. arr[start+seq_len : start+seq_len+predict_steps, target_idx]

Memory strategy: coordinate arrays are stored once; every sample is just a
(coord_idx, start) pointer. Sequences are sliced lazily in __getitem__, so we
never materialise the full 3-D tensor (which would blow up RAM for all of VN).
"""
import numpy as np
import torch
from torch.utils.data import Dataset

SEQUENCE_LENGTH = 168   # look back 7 days (168 hours)
PREDICT_STEPS   = 168   # forecast the next 7 days (168 hours) at once


# ── Coordinate array builder ──────────────────────────────────────────────

def build_coordinate_arrays(df, feature_cols: list):
    """
    Return:
      arrays : list of float32 arrays, one per (lat, lon), sorted by coordinate.
      coords : parallel list of (lat, lon) tuples (used by the spatial error map).
    """
    arrays, coords = [], []
    for (lat, lon), grp in df.groupby(['latitude', 'longitude'], sort=True):
        arrays.append(grp[feature_cols].values.astype(np.float32))
        coords.append((float(lat), float(lon)))
    return arrays, coords


# ── Temporal split (per coordinate, no cross-boundary contamination) ──────

def build_split_indices(
    coordinate_arrays: list,
    seq_len: int       = SEQUENCE_LENGTH,
    predict_steps: int = PREDICT_STEPS,
    train_ratio: float = 0.8,
):
    """
    Split each coordinate's time axis at 80%.
      Train samples : the whole window [start, start+seq_len+predict_steps) < split
      Test  samples : windows that start at/after split → no leakage from training.
    A sample needs seq_len + predict_steps consecutive hours to be valid.
    """
```

```
    span = seq_len + predict_steps
    train_idx, test_idx = [], []

    for cid, arr in enumerate(coordinate_arrays):
        n     = len(arr)
        split = int(n * train_ratio)

        # train: last index touched = start + span - 1 < split
        for start in range(0, max(0, split - span + 1)):
            train_idx.append((cid, start))

        # test: window starts at/after split, still fits inside n
        for start in range(split, max(split, n - span + 1)):
            test_idx.append((cid, start))

    return train_idx, test_idx


# ── Dataset ──────────────────────────────────────────────────────────

class WeatherSequenceDataset(Dataset):
    """
    Lazy multi-step sequence dataset. Each __getitem__ slices one (X, y) pair on
    the fly — no pre-materialised 3-D sequence tensor in RAM.

        X : (seq_len, n_features)
        y : (predict_steps,)        future rainfall (log1p + MinMax scaled)
    """

    def __init__(
        self,
        coordinate_arrays: list,
        indices: list,
        target_idx: int,
        seq_len: int       = SEQUENCE_LENGTH,
        predict_steps: int = PREDICT_STEPS,
    ):
        self.coordinate_arrays = coordinate_arrays
        self.indices           = indices
        self.target_idx        = target_idx
        self.seq_len           = seq_len
        self.predict_steps     = predict_steps

    def __len__(self) -> int:
        return len(self.indices)

    def __getitem__(self, idx):
        cid, start = self.indices[idx]
        arr = self.coordinate_arrays[cid]

        x_end = start + self.seq_len
        y_end = x_end + self.predict_steps

        # .copy() → independent buffers so DataLoader workers stay safe.
        x = arr[start:x_end].copy()
        y = arr[x_end:y_end, self.target_idx].copy()
```

```
        return torch.from_numpy(x), torch.from_numpy(y)
```

In [ ]:
```
%%writefile /kaggle/working/lstm_rain_weekly/model/__init__.py
# package marker
```

In [ ]:
```
%%writefile /kaggle/working/lstm_rain_weekly/model/lstm_model.py
"""
Task: Define the multi-step LSTM model.

Architecture (Direct Multi-step forecasting):
  LSTM(input_size, hidden=128, layers=2, dropout=0.1, batch_first)
  → take the hidden state of the LAST timestep
  → Linear(128, 64) → ReLU → Dropout(0.1) → Linear(64, 168)

The final layer emits all 168 future hours at once, so prediction errors are not
fed back into the model — this avoids the error accumulation of recursive
single-step forecasting.
"""
import torch
import torch.nn as nn


class LSTMModel(nn.Module):
    def __init__(
        self,
        input_size:  int,
        hidden_size: int   = 128,
        num_layers:  int   = 2,
        output_size: int   = 168,
        dropout:     float = 0.1,
    ):
        super().__init__()
        self.lstm = nn.LSTM(
            input_size  = input_size,
            hidden_size = hidden_size,
            num_layers  = num_layers,
            # PyTorch applies dropout between stacked LSTM layers only.
            dropout     = dropout if num_layers > 1 else 0.0,
            batch_first = True,
        )
        self.head = nn.Sequential(
            nn.Linear(hidden_size, 64),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(64, output_size),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # x: (batch, seq_len, input_size)
        out, _ = self.lstm(x)      # (batch, seq_len, hidden)
        out    = out[:, -1, :]     # last timestep → (batch, hidden)
        return self.head(out)      # (batch, output_size=168)
```

In [ ]:
```
%%writefile /kaggle/working/lstm_rain_weekly/training/__init__.py
# package marker
```

In [ ]:
```
%%writefile /kaggle/working/lstm_rain_weekly/training/trainer.py
"""
Task: Run the training loop — one epoch of gradient updates over the train set,
      followed by inference over the val set, logging MSE / RMSE / MAE for both.

Metrics here are computed in the model's working space (log1p + MinMax scaled),
which matches the optimisation objective. The real-unit (mm/hour) error is
reported separately by the evaluator via np.expm1 inversion.
"""
import numpy as np
import torch


# ── Metric helper ──────────────────────────────────────────────────────────

def _metrics(preds: np.ndarray, targets: np.ndarray):
    mse  = float(np.mean((preds - targets) ** 2))
    rmse = float(np.sqrt(mse))
    mae  = float(np.mean(np.abs(preds - targets)))
    return mse, rmse, mae


# ── Single epoch pass ──────────────────────────────────────────────────────

def _run_epoch(model, loader, optimizer, criterion, device, train: bool):
    model.train(train)
    sse = sae = count = 0.0   # streaming accumulators → no giant pred buffers

    with torch.set_grad_enabled(train):
        for X_b, y_b in loader:
            X_b = X_b.to(device, non_blocking=True)
            y_b = y_b.to(device, non_blocking=True)

            out  = model(X_b)              # (batch, 168)
            loss = criterion(out, y_b)

            if train:
                optimizer.zero_grad(set_to_none=True)
                loss.backward()
                optimizer.step()

            diff   = (out - y_b).detach()
            sse   += torch.sum(diff ** 2).item()
            sae   += torch.sum(torch.abs(diff)).item()
            count += diff.numel()

    mse  = sse / count
    rmse = float(np.sqrt(mse))
    mae  = sae / count
    return mse, rmse, mae
```

```python
# ── Full training loop ──────────────────────────────────────────────

def train_model(model, train_loader, val_loader, optimizer, criterion, device, epoc
    history = {k: [] for k in [
        'epoch',
        'train_mse', 'train_rmse', 'train_mae',
        'val_mse',   'val_rmse',   'val_mae',
    ]}

    for epoch in range(1, epochs + 1):
        tr_mse, tr_rmse, tr_mae = _run_epoch(
            model, train_loader, optimizer, criterion, device, train=True
        )
        vl_mse, vl_rmse, vl_mae = _run_epoch(
            model, val_loader, None, criterion, device, train=False
        )

        for key, val in zip(history.keys(), [
            epoch,
            tr_mse, tr_rmse, tr_mae,
            vl_mse, vl_rmse, vl_mae,
        ]):
            history[key].append(val)

        print(
            f"Epoch [{epoch:02d}/{epochs}]  "
            f"Train MSE: {tr_mse:.5f}, Val MSE: {vl_mse:.5f}  |  "
            f"Train RMSE: {tr_rmse:.5f}, Val RMSE: {vl_rmse:.5f}  |  "
            f"Train MAE: {tr_mae:.5f}, Val MAE: {vl_mae:.5f}"
        )

    return history
```

In [ ]:

```python
%%writefile /kaggle/working/lstm_rain_weekly/training/evaluator.py
"""
Task: Evaluate the trained model on the held-out test set and report errors in
      REAL rainfall units (mm/hour).

The target was trained as log1p(precip) then MinMax-scaled to [0, 1]. To get back
to mm/hour we invert in two steps:
    1. undo MinMax for the target column : x = scaled * data_range_ + data_min_
    2. undo the log                      : mm = np.expm1(x)
Both predictions and ground truth are inverted before computing RMSE / MAE.
"""
import numpy as np
import torch
from torch.utils.data import DataLoader, Subset

from dataset.sequence_dataset import WeatherSequenceDataset


# ── Target inverse transform (scaled-log → mm/hour) ─────────────────

def inverse_target(scaled, scaler, target_idx: int) -> np.ndarray:
    """Invert MinMax scaling then log1p for the rainfall target column."""
    scaled = np.asarray(scaled, dtype=np.float64)
```

```
    log_precip = scaled * scaler.data_range_[target_idx] + scaler.data_min_[target_
    return np.expm1(log_precip)


# ── Test-set metrics (streaming, in mm/hour) ──────────────────────────────

def evaluate_test(model, test_dataset, device, scaler, target_idx: int,
                  batch_size: int = 2048) -> tuple:
    loader = DataLoader(
        test_dataset, batch_size=batch_size, shuffle=False,
        num_workers=2, pin_memory=(device.type == 'cuda'),
    )

    model.eval()
    sse = sae = count = 0.0

    with torch.no_grad():
        for X_b, y_b in loader:
            X_b  = X_b.to(device, non_blocking=True)
            out  = model(X_b).cpu().numpy()

            preds_mm   = inverse_target(out,          scaler, target_idx)
            targets_mm = inverse_target(y_b.numpy(),  scaler, target_idx)

            diff   = preds_mm - targets_mm
            sse   += float(np.sum(diff ** 2))
            sae   += float(np.sum(np.abs(diff)))
            count += diff.size

    mse  = sse / count
    rmse = float(np.sqrt(mse))
    mae  = sae / count

    w = 56
    print()
    print("=" * w)
    print(f"{'TEST SET EVALUATION  (real units, mm/hour)':^{w}}")
    print("=" * w)
    print(f"  {'Metric':<26} {'Value':>16}")
    print("-" * w)
    print(f"  {'Test MSE  (mm^2/h)':<26} {mse:>16.6f}")
    print(f"  {'Test RMSE (mm/hour)':<26} {rmse:>16.6f}")
    print(f"  {'Test MAE  (mm/hour)':<26} {mae:>16.6f}")
    print("=" * w)

    return mse, rmse, mae


# ── Prediction collector for visualisation (subsampled, in mm/hour) ───────

def collect_predictions(model, coordinate_arrays, indices, target_idx, scaler,
                        seq_len, predict_steps, device,
                        max_samples: int = 4000, batch_size: int = 2048):
    """
    Run inference over a random subset of `indices` and return real-unit arrays
    for plotting:
```

```
        preds_mm   : (M, predict_steps)
        targets_mm : (M, predict_steps)
        coord_ids  : (M,)  coordinate index of each sample
    Subsampled to keep the plotting buffers small while covering many regions.
    """
    rng = np.random.default_rng(42)
    if len(indices) > max_samples:
        pick = rng.choice(len(indices), size=max_samples, replace=False)
        pick.sort()
        sub_indices = [indices[i] for i in pick]
    else:
        sub_indices = list(indices)

    ds = WeatherSequenceDataset(coordinate_arrays, sub_indices, target_idx,
                                seq_len, predict_steps)
    loader = DataLoader(ds, batch_size=batch_size, shuffle=False,
                        num_workers=2, pin_memory=(device.type == 'cuda'))

    model.eval()
    preds_buf, tgts_buf = [], []
    with torch.no_grad():
        for X_b, y_b in loader:
            X_b = X_b.to(device, non_blocking=True)
            preds_buf.append(model(X_b).cpu().numpy())
            tgts_buf.append(y_b.numpy())

    preds_mm   = inverse_target(np.concatenate(preds_buf), scaler, target_idx)
    targets_mm = inverse_target(np.concatenate(tgts_buf),  scaler, target_idx)
    coord_ids  = np.array([cid for cid, _ in sub_indices], dtype=np.int64)

    return preds_mm, targets_mm, coord_ids
```

In [ ]:
```
%%writefile /kaggle/working/lstm_rain_weekly/visualization/__init__.py
# package marker
```

In [ ]:
```
%%writefile /kaggle/working/lstm_rain_weekly/visualization/plot_loss.py
"""
Task: Persist the training history CSV and render the 8 report-grade figures into
      the 'plots/' directory.

All rainfall quantities passed in here are already in REAL units (mm/hour) — the
caller inverts log1p + MinMax via training.evaluator.inverse_target first.

Figures
  1-3. plots/loss_mse.png, loss_rmse.png, loss_mae.png  — Train vs Val curves.
  4.   plots/scatter_actual_vs_pred.png                 — actual vs predicted (+ R²
  5.   plots/sample_prediction_comparison.png           — one coord, 168h actual vs
  6.   plots/residuals_histogram.png                    — distribution of (actual -
  7.   plots/feature_correlation_heatmap.png            — input-feature correlation
  8.   plots/spatial_error_heatmap.png                  — per-coordinate MAE map of
  9.   plots/feature_distributions.png                  — rainfall raw vs log1p.
  10.  plots/residuals_vs_fitted.png                    — residuals vs fitted value
"""
import os
import numpy as np
```

```python
import pandas as pd

import matplotlib
matplotlib.use('Agg')   # headless backend — safe inside Kaggle notebooks
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import r2_score

PLOTS_DIR        = 'plots'
HISTORY_CSV_PATH = 'training_history_rain.csv'

sns.set_theme(style='whitegrid')
_RNG = np.random.default_rng(7)


# ── Infrastructure ──────────────────────────────────────────────────────────

def ensure_plots_dir(path: str = PLOTS_DIR) -> str:
    os.makedirs(path, exist_ok=True)
    return path


def save_history(history: dict, save_path: str = HISTORY_CSV_PATH) -> None:
    pd.DataFrame(history).to_csv(save_path, index=False)
    print(f"  Training history -> {save_path}")


def _subsample_flat(*arrays, n: int = 25000):
    """Flatten then take a shared random subset of points (for scatter/hist)."""
    flat = [a.ravel() for a in arrays]
    total = flat[0].size
    if total > n:
        pick = _RNG.choice(total, size=n, replace=False)
        flat = [f[pick] for f in flat]
    return flat


# ── 1-3. Loss curves ────────────────────────────────────────────────────────

def plot_loss_curves(history: dict, out_dir: str = PLOTS_DIR) -> None:
    epochs = history['epoch']
    specs = [
        ('mse',  'MSE',  'loss_mse.png'),
        ('rmse', 'RMSE', 'loss_rmse.png'),
        ('mae',  'MAE',  'loss_mae.png'),
    ]
    for key, label, fname in specs:
        fig, ax = plt.subplots(figsize=(10, 5))
        ax.plot(epochs, history[f'train_{key}'], 'o-',  label=f'Train {label}', lin
        ax.plot(epochs, history[f'val_{key}'],   's--', label=f'Val {label}',   lin
        ax.set_xlabel('Epoch')
        ax.set_ylabel(f'{label} (scaled-log space)')
        ax.set_title(f'LSTM Weekly Rain — Train vs Val {label}')
        ax.legend()
        ax.set_xticks(epochs)
        fig.tight_layout()
```

```
        fig.savefig(os.path.join(out_dir, fname), dpi=150)
        plt.close(fig)
        print(f"  Loss curve       -> {os.path.join(out_dir, fname)}")


# ── 4. Scatter actual vs predicted ──────────────────────────────────

def plot_scatter_actual_vs_pred(targets_mm, preds_mm, out_dir: str = PLOTS_DIR) ->
    a, p = _subsample_flat(targets_mm, preds_mm)
    r2 = r2_score(targets_mm.ravel(), preds_mm.ravel())

    lim = max(a.max(), p.max()) * 1.02
    fig, ax = plt.subplots(figsize=(7, 7))
    ax.scatter(a, p, s=6, alpha=0.25, edgecolors='none')
    ax.plot([0, lim], [0, lim], 'r--', linewidth=2, label='Ideal  y = x')
    ax.set_xlim(0, lim); ax.set_ylim(0, lim)
    ax.set_xlabel('Actual rainfall (mm/hour)')
    ax.set_ylabel('Predicted rainfall (mm/hour)')
    ax.set_title(f'Actual vs Predicted Rainfall  (R² = {r2:.3f})')
    ax.legend()
    fig.tight_layout()
    fig.savefig(os.path.join(out_dir, 'scatter_actual_vs_pred.png'), dpi=150)
    plt.close(fig)
    print(f"  Scatter A/P      -> {os.path.join(out_dir, 'scatter_actual_vs_pred.pn


# ── 5. Sample 168h forecast vs actual (one coordinate) ──────────────

def plot_sample_prediction(targets_mm, preds_mm, coord_ids, coords,
                           out_dir: str = PLOTS_DIR) -> None:
    i = int(_RNG.integers(len(preds_mm)))
    lat, lon = coords[coord_ids[i]]
    hours = np.arange(targets_mm.shape[1])

    fig, ax = plt.subplots(figsize=(13, 5))
    ax.plot(hours, targets_mm[i], color='royalblue', linewidth=2,
            label='Thực tế (actual)')
    ax.plot(hours, preds_mm[i], color='deeppink', linestyle='--', linewidth=2,
            label='Dự báo AI (forecast)')
    ax.set_xlabel('Giờ tương lai (0 → 168h)')
    ax.set_ylabel('Lượng mưa (mm/hour)')
    ax.set_title(f'Dự báo 1 tuần tới tại tọa độ (lat={lat:.2f}, lon={lon:.2f})')
    ax.legend()
    fig.tight_layout()
    fig.savefig(os.path.join(out_dir, 'sample_prediction_comparison.png'), dpi=150)
    plt.close(fig)
    print(f"  Sample forecast  -> {os.path.join(out_dir, 'sample_prediction_compari


# ── 6. Residuals histogram ──────────────────────────────────────────

def plot_residuals_histogram(targets_mm, preds_mm, out_dir: str = PLOTS_DIR) -> Non
    resid = (targets_mm - preds_mm).ravel()
    (resid,) = _subsample_flat(resid, n=60000)

    fig, ax = plt.subplots(figsize=(9, 5))
```

```
    sns.histplot(resid, bins=80, kde=True, ax=ax, color='steelblue')
    ax.axvline(0, color='red', linestyle='--', linewidth=2, label='Zero error')
    ax.set_xlabel('Residual = Actual − Predicted (mm/hour)')
    ax.set_ylabel('Frequency')
    ax.set_title(f'Residual Distribution  (mean = {resid.mean():.4f} mm/h)')
    ax.legend()
    fig.tight_layout()
    fig.savefig(os.path.join(out_dir, 'residuals_histogram.png'), dpi=150)
    plt.close(fig)
    print(f"  Residual hist    -> {os.path.join(out_dir, 'residuals_histogram.png')


# ── 7. Feature correlation heatmap ──────────────────────────────────────

def plot_feature_correlation_heatmap(df_sample, feature_cols,
                                     target_col: str = 'total_precipitation',
                                     out_dir: str = PLOTS_DIR) -> None:
    cols = [c for c in feature_cols if c in df_sample.columns]
    corr = df_sample[cols].corr()

    fig, ax = plt.subplots(figsize=(14, 12))
    sns.heatmap(corr, cmap='coolwarm', center=0, square=True,
                linewidths=0.4, cbar_kws={'shrink': 0.8}, ax=ax)
    ax.set_title(f'Input Feature Correlation Matrix (target: {target_col})')
    fig.tight_layout()
    fig.savefig(os.path.join(out_dir, 'feature_correlation_heatmap.png'), dpi=150)
    plt.close(fig)
    print(f"  Corr heatmap     -> {os.path.join(out_dir, 'feature_correlation_heatm


# ── 8. Spatial error heatmap ────────────────────────────────────────────

def plot_spatial_error_heatmap(targets_mm, preds_mm, coord_ids, coords,
                               out_dir: str = PLOTS_DIR) -> None:
    abs_err = np.abs(targets_mm - preds_mm).mean(axis=1)   # per-sample MAE

    lats, lons, maes = [], [], []
    for cid in np.unique(coord_ids):
        mask = coord_ids == cid
        lat, lon = coords[cid]
        lats.append(lat); lons.append(lon)
        maes.append(float(abs_err[mask].mean()))

    fig, ax = plt.subplots(figsize=(8, 10))
    sc = ax.scatter(lons, lats, c=maes, cmap='YlOrRd', s=60,
                    edgecolors='k', linewidths=0.3)
    fig.colorbar(sc, ax=ax, label='Mean Absolute Error (mm/hour)')
    ax.set_xlabel('Longitude'); ax.set_ylabel('Latitude')
    ax.set_title('Spatial Forecast Error Across Vietnam')
    fig.tight_layout()
    fig.savefig(os.path.join(out_dir, 'spatial_error_heatmap.png'), dpi=150)
    plt.close(fig)
    print(f"  Spatial error    -> {os.path.join(out_dir, 'spatial_error_heatmap.png


# ── 9. Rainfall distribution: raw vs log1p ──────────────────────────────
```

```python
def plot_feature_distributions(raw_precip, out_dir: str = PLOTS_DIR) -> None:
    raw = np.asarray(raw_precip, dtype=np.float64)
    raw = raw[np.isfinite(raw)]
    raw = np.clip(raw, 0, None)
    logged = np.log1p(raw)

    fig, axes = plt.subplots(1, 2, figsize=(13, 5))
    sns.histplot(raw, bins=80, ax=axes[0], color='indianred')
    axes[0].set_title('Raw total_precipitation (right-skewed)')
    axes[0].set_xlabel('mm/hour'); axes[0].set_ylabel('Frequency')

    sns.histplot(logged, bins=80, ax=axes[1], color='seagreen')
    axes[1].set_title('log1p(total_precipitation) (compressed tail)')
    axes[1].set_xlabel('log1p(mm/hour)'); axes[1].set_ylabel('Frequency')

    fig.suptitle('Why we log-transform rainfall before training')
    fig.tight_layout()
    fig.savefig(os.path.join(out_dir, 'feature_distributions.png'), dpi=150)
    plt.close(fig)
    print(f"  Distributions    -> {os.path.join(out_dir, 'feature_distributions.png

# ── 10. Residuals vs fitted ──────────────────────────────────────────

def plot_residuals_vs_fitted(targets_mm, preds_mm, out_dir: str = PLOTS_DIR) -> Non
    fitted, resid = _subsample_flat(preds_mm, targets_mm - preds_mm)

    fig, ax = plt.subplots(figsize=(9, 6))
    ax.scatter(fitted, resid, s=6, alpha=0.25, edgecolors='none')
    ax.axhline(0, color='red', linestyle='--', linewidth=2)
    ax.set_xlabel('Fitted / predicted rainfall (mm/hour)')
    ax.set_ylabel('Residual = Actual − Predicted (mm/hour)')
    ax.set_title('Residuals vs Fitted Values')
    fig.tight_layout()
    fig.savefig(os.path.join(out_dir, 'residuals_vs_fitted.png'), dpi=150)
    plt.close(fig)
    print(f"  Resid vs fitted  -> {os.path.join(out_dir, 'residuals_vs_fitted.png')


# ── Orchestrator ─────────────────────────────────────────────────────

def save_all_plots(history, targets_mm, preds_mm, coord_ids, coords,
                   df_sample, feature_cols, raw_precip,
                   target_col: str = 'total_precipitation',
                   out_dir: str = PLOTS_DIR) -> None:
    ensure_plots_dir(out_dir)
    plot_loss_curves(history, out_dir)
    plot_scatter_actual_vs_pred(targets_mm, preds_mm, out_dir)
    plot_sample_prediction(targets_mm, preds_mm, coord_ids, coords, out_dir)
    plot_residuals_histogram(targets_mm, preds_mm, out_dir)
    plot_feature_correlation_heatmap(df_sample, feature_cols, target_col, out_dir)
    plot_spatial_error_heatmap(targets_mm, preds_mm, coord_ids, coords, out_dir)
    plot_feature_distributions(raw_precip, out_dir)
    plot_residuals_vs_fitted(targets_mm, preds_mm, out_dir)
```

In [ ]:

```
%%writefile /kaggle/working/lstm_rain_weekly/main.py
"""
Task: Orchestrate the full weekly-rainfall pipeline:
      Load → Engineer → Scale → Sequences → Train → Evaluate → 8 Plots → Checkpoint

Run on Kaggle (GPU):
    !python /kaggle/working/lstm_rain_weekly/main.py
"""
import os
import sys

# Allow sibling-package imports regardless of the current working directory.
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader

from data.load_data import load_data
from preprocessing.feature_engineering import engineer_features, TARGET_COL
from preprocessing.scaling import fit_and_scale, SCALER_PATH, FEATURE_COLS_PATH
from dataset.sequence_dataset import (
    build_coordinate_arrays,
    build_split_indices,
    WeatherSequenceDataset,
    SEQUENCE_LENGTH,
    PREDICT_STEPS,
)
from model.lstm_model import LSTMModel
from training.trainer import train_model
from training.evaluator import evaluate_test, collect_predictions
from visualization.plot_loss import save_history, save_all_plots

# ── Hyper-parameters ──────────────────────────────────────────────────────────
EPOCHS      = 15
BATCH_SIZE  = 2048      # large batch to saturate the Kaggle T4 GPU
HIDDEN_SIZE = 128
NUM_LAYERS  = 2
DROPOUT     = 0.1
LR          = 0.001
OUTPUT_SIZE = PREDICT_STEPS          # 168 future hours (Direct Multi-step)

MODELS_DIR  = 'models'
MODEL_PATH  = os.path.join(MODELS_DIR, 'lstm_rain_weekly_model.pt')


def main() -> None:
    # ── Device ────────────────────────────────────────────────────────────────
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    print(f"Device : {device}")
    if device.type == 'cuda':
        print(f"GPU    : {torch.cuda.get_device_name(0)}")
    use_pin = device.type == 'cuda'
```

```python
# ── 1. Load ──────────────────────────────────────────────────────────
print("\n[1/7] Loading data ...")
df = load_data()

# Keep a raw-rainfall sample BEFORE log1p — needed for the distribution plot.
raw_precip = df[TARGET_COL].sample(
    n=min(100_000, len(df)), random_state=0
).to_numpy()

# ── 2. Feature engineering ──────────────────────────────────────────
print("\n[2/7] Engineering features ...")
df = engineer_features(df)
print(f"  {len(df):>12,} rows  |  {len(df.columns)} columns")

# ── 3. Scale ─────────────────────────────────────────────────────────
print("\n[3/7] Scaling ...")
df, scaler, feature_cols = fit_and_scale(df)
target_idx = feature_cols.index(TARGET_COL)

# Sample of the scaled frame for the correlation heatmap (before we drop df).
df_sample = df[feature_cols].sample(
    n=min(50_000, len(df)), random_state=0
).copy()

# ── 4. Sequences ─────────────────────────────────────────────────────
print("\n[4/7] Building sequences ...")
coordinate_arrays, coords = build_coordinate_arrays(df, feature_cols)
del df   # release the big frame; arrays + sample are all we still need

train_idx, test_idx = build_split_indices(
    coordinate_arrays, SEQUENCE_LENGTH, PREDICT_STEPS
)
print(f"  Train: {len(train_idx):,}  |  Test: {len(test_idx):,}  "
      f"(seq_len={SEQUENCE_LENGTH}, predict_steps={PREDICT_STEPS})")

train_ds = WeatherSequenceDataset(
    coordinate_arrays, train_idx, target_idx, SEQUENCE_LENGTH, PREDICT_STEPS
)
test_ds = WeatherSequenceDataset(
    coordinate_arrays, test_idx, target_idx, SEQUENCE_LENGTH, PREDICT_STEPS
)

train_loader = DataLoader(
    train_ds, batch_size=BATCH_SIZE, shuffle=True,
    num_workers=2, pin_memory=use_pin, persistent_workers=True,
)
# The test split doubles as the per-epoch validation monitor.
val_loader = DataLoader(
    test_ds, batch_size=BATCH_SIZE, shuffle=False,
    num_workers=2, pin_memory=use_pin, persistent_workers=True,
)

# ── 5. Model ─────────────────────────────────────────────────────────
input_size = len(feature_cols)
model      = LSTMModel(input_size, HIDDEN_SIZE, NUM_LAYERS, OUTPUT_SIZE, DROPOU
optimizer  = optim.Adam(model.parameters(), lr=LR)
```

```
    criterion  = nn.MSELoss()

    n_params = sum(p.numel() for p in model.parameters())
    print(f"\n[5/7] Model ready  |  input_size={input_size}  |  "
          f"output_size={OUTPUT_SIZE}  |  params={n_params:,}")

    # ── 6. Train ──────────────────────────────────────────────────────────
    print(f"\n[6/7] Training  ({EPOCHS} epochs, batch={BATCH_SIZE}) ...")
    print("-" * 100)
    history = train_model(
        model, train_loader, val_loader, optimizer, criterion, device, EPOCHS
    )
    print("-" * 100)

    # ── 7. Evaluate (real units) + visualise + export ─────────────────────
    print("\n[7/7] Evaluating on the test set ...")
    evaluate_test(model, test_ds, device, scaler, target_idx, BATCH_SIZE)

    print("\n[Export]")
    save_history(history)

    preds_mm, targets_mm, coord_ids = collect_predictions(
        model, coordinate_arrays, test_idx, target_idx, scaler,
        SEQUENCE_LENGTH, PREDICT_STEPS, device,
        max_samples=4000, batch_size=BATCH_SIZE,
    )
    save_all_plots(
        history, targets_mm, preds_mm, coord_ids, coords,
        df_sample, feature_cols, raw_precip, target_col=TARGET_COL,
    )

    # ── Checkpoint ────────────────────────────────────────────────────────
    os.makedirs(MODELS_DIR, exist_ok=True)
    torch.save(
        {
            'model_state_dict':  model.state_dict(),
            'input_size':        input_size,
            'hidden_size':       HIDDEN_SIZE,
            'num_layers':        NUM_LAYERS,
            'output_size':       OUTPUT_SIZE,        # 168
            'dropout':           DROPOUT,
            'sequence_length':   SEQUENCE_LENGTH,    # 168
            'target_cols':       [TARGET_COL],       # ['total_precipitation']
            'scaler_name':       SCALER_PATH,        # 'scaler_rain.pkl'
            'feature_cols_name': FEATURE_COLS_PATH,  # 'feature_cols_rain.pkl'
        },
        MODEL_PATH,
    )
    print(f"  Checkpoint       -> {MODEL_PATH}")
    print("\n[Done]")


if __name__ == '__main__':
    main()
```

Huấn luyện

Chạy toàn bộ pipeline: Load → Engineer → Scale → Sequences → Train → Evaluate → 8 Plots → Checkpoint.

In [ ]:
```
!python /kaggle/working/lstm_rain_weekly/main.py
```

Xem 8 biểu đồ kết quả

In [ ]:
```
from IPython.display import Image, display
import os

plot_files = [
    'loss_mse.png', 'loss_rmse.png', 'loss_mae.png',
    'scatter_actual_vs_pred.png', 'sample_prediction_comparison.png',
    'residuals_histogram.png', 'feature_correlation_heatmap.png',
    'spatial_error_heatmap.png', 'feature_distributions.png',
    'residuals_vs_fitted.png',
]
for name in plot_files:
    path = os.path.join('plots', name)
    if os.path.exists(path):
        print(name)
        display(Image(filename=path))
```